A* — Misplaced Tiles

```
import heapq

# Initial and goal states
initial = (
    2, 8, 3,
    1, 6, 4,
    7, 0, 5
)

goal = (
    1, 2, 3,
    8, 0, 4,
    7, 6, 5
)


# Display puzzle
def display(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()


# Heuristic: Number of misplaced tiles
def misplaced_tiles(state):
    count = 0

    for i in range(9):
        if state[i] != 0 and state[i] != goal[i]:
            count += 1

    return count


# Generate possible next states
def get_neighbors(state):
    neighbors = []

    blank = state.index(0)
    row = blank // 3
    col = blank % 3
```

```
    moves = []

    if row > 0:
        moves.append(blank - 3)       # Up

    if row < 2:
        moves.append(blank + 3)       # Down

    if col > 0:
        moves.append(blank - 1)       # Left

    if col < 2:
        moves.append(blank + 1)       # Right

    for move in moves:
        new_state = list(state)

        new_state[blank], new_state[move] = \
            new_state[move], new_state[blank]

        neighbors.append(tuple(new_state))

    return neighbors


# A* Search
def a_star(initial, goal):

    # Priority queue:
    # (f, g, state, path)
    open_list = []

    h = misplaced_tiles(initial)

    heapq.heappush(
        open_list,
        (h, 0, initial, [initial])
    )

    visited = set()

    while open_list:

        f, g, current, path = heapq.heappop(open_list)
```

```
        if current in visited:
            continue

        visited.add(current)

        print("Current State:")
        display(current)

        print("g(n) =", g)
        print("h(n) =", misplaced_tiles(current))
        print("f(n) =", f)
        print("----------------------")

        # Goal reached
        if current == goal:
            print("\nGoal Reached!")
            print("Total Moves =", g)

            print("\nSolution Path:")
            for state in path:
                display(state)

            return

        # Generate successors
        for neighbor in get_neighbors(current):

            if neighbor not in visited:

                new_g = g + 1
                new_h = misplaced_tiles(neighbor)
                new_f = new_g + new_h

                heapq.heappush(
                    open_list,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    print("No solution found.")


# Run A*
a_star(initial, goal)
```

```
----------------------
Current State:
(0, 2, 3)
(1, 8, 4)
(7, 6, 5)

g(n) = 3
h(n) = 2
f(n) = 5
----------------------
Current State:
(1, 2, 3)
(0, 8, 4)
(7, 6, 5)

g(n) = 4
h(n) = 1
f(n) = 5
----------------------
Current State:
(1, 2, 3)
(8, 0, 4)
(7, 6, 5)

g(n) = 5
h(n) = 0
f(n) = 5
----------------------

Goal Reached!
Total Moves = 5

Solution Path:
(2, 8, 3)
(1, 6, 4)
(7, 0, 5)

(2, 8, 3)
(1, 0, 4)
(7, 6, 5)

(2, 0, 3)
(1, 8, 4)
(7, 6, 5)

(0, 2, 3)
(1, 8, 4)
(7, 6, 5)

(1, 2, 3)
(0, 8, 4)
(7, 6, 5)

(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
```

A* — Manhattan Distance

```
import heapq

# Initial and goal states
initial = (
    2, 8, 3,
    1, 6, 4,
    7, 0, 5
)

goal = (
    1, 2, 3,
    8, 0, 4,
    7, 6, 5
)


# Display puzzle
def display(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()


# Heuristic: Manhattan Distance
def manhattan_distance(state):

    distance = 0

    for i in range(9):

        # Do not consider blank
        if state[i] == 0:
            continue

        # Current position
        current_row = i // 3
        current_col = i % 3

        # Goal position
        goal_index = goal.index(state[i])
        goal_row = goal_index // 3
        goal_col = goal_index % 3
```

```
            # Manhattan distance
            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)

    return distance


# Generate possible next states
def get_neighbors(state):

    neighbors = []

    blank = state.index(0)

    row = blank // 3
    col = blank % 3

    moves = []

    if row > 0:
        moves.append(blank - 3)       # Up

    if row < 2:
        moves.append(blank + 3)       # Down

    if col > 0:
        moves.append(blank - 1)       # Left

    if col < 2:
        moves.append(blank + 1)       # Right

    for move in moves:

        new_state = list(state)

        new_state[blank], new_state[move] = \
            new_state[move], new_state[blank]

        neighbors.append(tuple(new_state))

    return neighbors


# A* Search
```

```
def a_star(initial, goal):

    # Priority queue:
    # (f, g, state, path)
    open_list = []

    h = manhattan_distance(initial)

    heapq.heappush(
        open_list,
        (h, 0, initial, [initial])
    )

    visited = set()

    while open_list:

        f, g, current, path = heapq.heappop(open_list)

        if current in visited:
            continue

        visited.add(current)

        print("Current State:")
        display(current)

        print("g(n) =", g)
        print("h(n) =", manhattan_distance(current))
        print("f(n) =", f)
        print("----------------------")

        # Goal reached
        if current == goal:

            print("\nGoal Reached!")
            print("Total Moves =", g)

            print("\nSolution Path:")

            for state in path:
                display(state)

            return
```

```
        # Generate successors
        for neighbor in get_neighbors(current):

            if neighbor not in visited:

                new_g = g + 1
                new_h = manhattan_distance(neighbor)
                new_f = new_g + new_h

                heapq.heappush(
                    open_list,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    print("No solution found.")


# Run A*
a_star(initial, goal)
```

```
----------------------
Current State:
(0, 2, 3)
(1, 8, 4)
(7, 6, 5)

g(n) = 3
h(n) = 2
f(n) = 5
----------------------
Current State:
(1, 2, 3)
(0, 8, 4)
(7, 6, 5)

g(n) = 4
h(n) = 1
f(n) = 5
----------------------
Current State:
(1, 2, 3)
(8, 0, 4)
(7, 6, 5)

g(n) = 5
h(n) = 0
f(n) = 5
----------------------

Goal Reached!
Total Moves = 5

Solution Path:
(2, 8, 3)
(1, 6, 4)
(7, 0, 5)

(2, 8, 3)
(1, 0, 4)
(7, 6, 5)

(2, 0, 3)
(1, 8, 4)
(7, 6, 5)

(0, 2, 3)
(1, 8, 4)
(7, 6, 5)

(1, 2, 3)
(0, 8, 4)
(7, 6, 5)

(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
```